

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

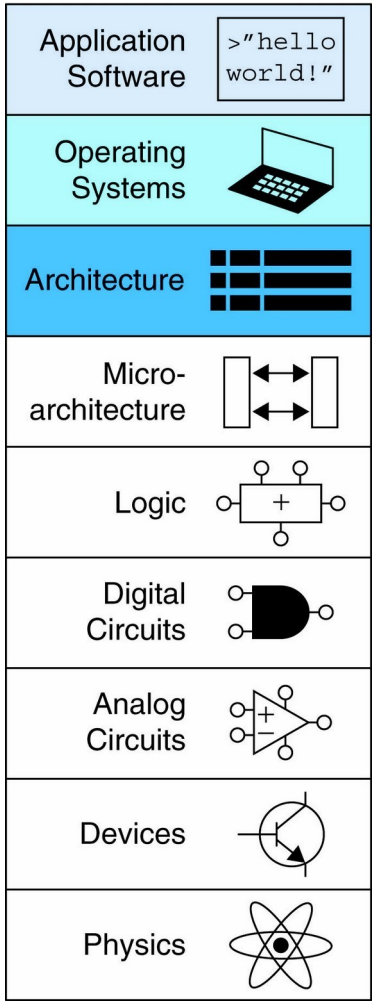
Lights, Camera, Action: Compiling, Assembling, and Loading

TEAM DLCD
Department of CSE and CSE (AI & ML)

DIGITAL LOGIC & COMPUTER DESIGN

TOPICS

- **Lights, Camera, Action: Compiling, Assembling, and Loading**



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Lights, Camera, Action: Compiling, Assembling, and Loading

- Until now: translating short high-level snippets into assembly and machine code.
- This section: compiling and assembling a complete program, and loading it into memory for execution.
- We will study about the MIPS memory map (where code, data and stack live) and the steps of code execution for a sample program in this section.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

The MIPS Memory Map

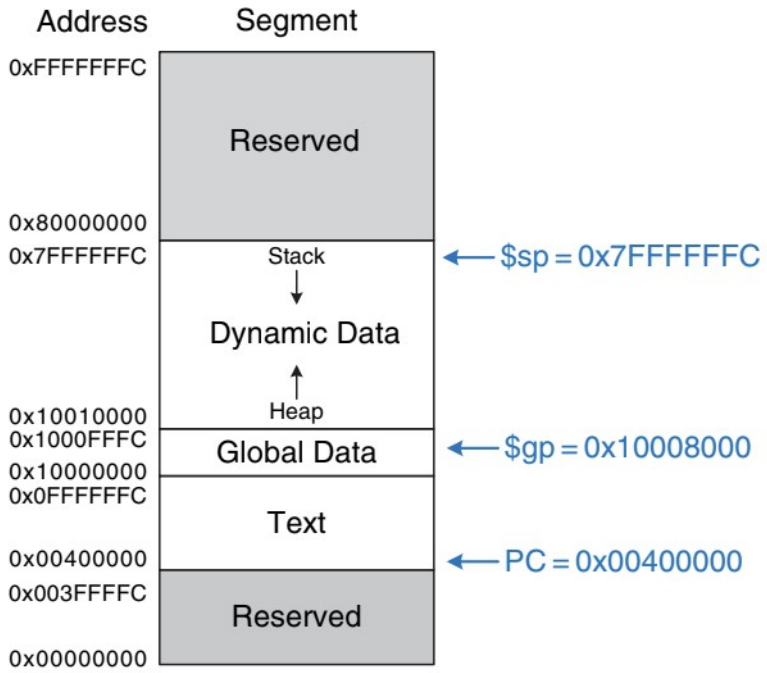
- 32-bit addresses → address space = 2^{32} bytes = **4 GB**.
- Word addresses are divisible by 4, ranging from 0 to 0xFFFFFFFC.
- Four segments: **text, global data, dynamic data, reserved**.

Address range	Segment
0x00000000 – 0x003FFFFC	Reserved
0x00400000 – 0x0FFFFFFC	Text (PC starts at 0x00400000)
0x10000000 – 0x1000FFFC	Global data ($gp = 0x10008000)
0x10010000 – 0x7FFFFFFC	Dynamic data: heap (bottom) and stack (top, $sp = 0x7FFFFFFC)
0x80000000 – 0xFFFFFFFC	Reserved

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

The MIPS Memory Map



MIPS memory map

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Text Segment

- Stores the **machine language program**.
- Large enough for almost **256 MB** of code.
- The top 4 bits of every text address are all 0, so the **j instruction can jump directly to any address** in the program.

Analogy: The text segment is the recipe book. The PC is your finger on the line you are currently following. The book is read-only reference material that tells the CPU what to do.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Global Data Segment

- Stores **global variables**, which, unlike local variables, can be seen by all functions.
- These are defined at **start-up**, before the program begins executing.
- These variables are declared outside main in C and accessible by any function.
- Large enough for **64 KB** of globals.
- **Accessing Globals with $gp:** Global variables are accessed using the global pointer ($gp), which is initialized to 0x100080000. Unlike the stack pointer ($sp), $gp does not change during program execution. Any global variable can be accessed with a **16-bit positive or negative offset from $gp**. The offset is known at assembly time, so the variables can be efficiently accessed using base addressing mode with constant offsets.

Analogy: Global variables are the noticeboard in a shared office, readable and writable by everyone. Local variables are notes on your own desk.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Dynamic Data Segment (Stack and Heap)

- Holds the **stack and the heap**.
- Data is not known at start-up; it is **allocated and deallocated during execution**.
- This is the largest segment, spanning almost **2 GB**.

	Stack	Heap
Use	Save/restore registers used by functions; hold local variables such as arrays	Data allocated at runtime
Allocation	LIFO stack frames	**malloc** in C; **new** in C++ and Java
Direction	Grows **downward** from the top (0x7FFFFFFC)	Grows **upward** from the bottom
Order	Last-in-first-out	Used and discarded in any order

- If stack and heap grow into each other, program data can be **corrupted**. The memory allocator prevents this by returning an **out-of-memory error** when space is insufficient.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Reserved Segments

- Used by the **operating system**; cannot be used directly by the program.
- Part of the reserved memory is used for **interrupts** and **memory-mapped I/O** (to be discussed in depth in future lectures).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

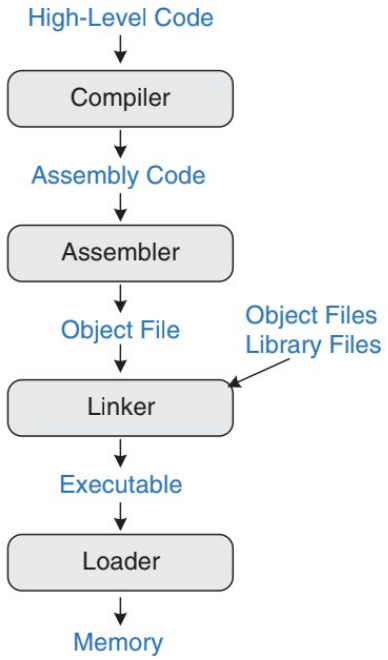
DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program (Overview)

High-level code → Compiler → Assembly code → Assembler → Object file → Linker (+ object files and library files) → Executable → Loader → Memory

1. **Compile**: high-level code → assembly code.
2. **Assemble**: assembly → machine code in an **object file**.
3. **Link**: combine machine code with library/other object code → complete **executable**.
4. **Load**: loader puts the program in memory and starts execution.

In practice, most compilers perform compiling, assembling and linking together.



Steps for translating and starting a program

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 1 - Compilation

A compiler translates high-level code into assembly.

Example:

High-Level Code (C language):

```
int f, g, y; // global variables
int main(void) {
f = 2;
g = 3;
y = sum(f, g);
return y; }
int sum(int a, int b) {
return (a + b); }
```

MIPS assembly:

```
.data
f:
g:
y:
.text
main:
  addi $sp, $sp, -4   # make stack frame
  sw   $ra, 0($sp)    # store $ra on stack
  addi $a0, $0, 2     # $a0 = 2
  sw   $a0, f         # f = 2
  addi $a1, $0, 3     # $a1 = 3
  sw   $a1, g         # g = 3
  jal  sum            # call sum function
  sw   $v0, y         # y = sum(f, g)
  lw   $ra, 0($sp)    # restore $ra from stack
  addi $sp, $sp, 4    # restore stack pointer
  jr   $ra            # return to operating system
sum:
  add  $v0, $a0, $a1  # $v0 = a+b
  jr   $ra            # return to caller
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Translating and Starting a Program: Step 1 - Compilation

- .data and .text are **assembler directives** marking where the data and text segments begin.
- Labels f, g, y name the globals. Their storage locations are decided by the assembler; for now they stay as **symbols**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 2 - Assembling (Pass 1)

- The assembler converts assembly into an **object file** containing machine code.
- It makes **two passes**.
- **Pass 1:** assign instruction addresses and find all symbols (labels, global variable names).

Address	Instruction
0x00400000	main: addi $sp, $sp, -4
0x00400004	sw $ra, 0($sp)
0x00400008	addi $a0, $0, 2
0x0040000C	sw $a0, f
0x00400010	addi $a1, $0, 3
0x00400014	sw $a1, g
0x00400018	jal sum

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 2 - Assembling (Pass 1)

0x0040001C	sw $v0, y
0x00400020	lw $ra, 0($sp)
0x00400024	addi $sp, $sp, 4
0x00400028	jr $ra
0x0040002C	sum: add $v0, $a0, $a1
0x00400030	jr $ra

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 2 - Assembling (Symbol Table and Pass 2)

- Names and addresses of symbols are kept in the symbol table as shown in the table; addresses are filled in after Pass 1, when label addresses are known.
- Global variables are assigned storage in the global data segment **starting at 0x10000000**.
- **Pass 2:** the assembler produces machine code, taking addresses of globals and labels from the symbol table.
- The machine code and symbol table are stored in the object file.

Symbol	Address
f	0x10000000
g	0x10000004
y	0x10000008
main	0x00400000
sum	0x0040002C

Symbol table

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 3 - Linking

Why link?

- Large programs have many files. If only one changes, recompiling and reassembling the others is wasteful.
- Library files almost never change.
- If a high-level file is unchanged, its object file need not be updated.

What the linker does

- Combines all object files into one machine-language file: the **executable**.
- **Relocates** data and instructions so they are not all on top of each other.
- Uses the symbol tables to adjust addresses of relocated globals and labels.
- In our example there is one object file, so **no relocation** is needed.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: The Executable

Executable has three sections: **header, text segment, data segment.**

- **Header:** text size = **0x34 (52 bytes)** (13 instructions × 4), data size = **0xC (12 bytes)** (3 words).
- **Text segment:** instructions in the order stored in memory.
- **Data segment:** address of each global variable.
- The figure shows readable instructions beside the machine code for convenience; the real file has **only machine instructions**.

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	**Address**	**Instruction**
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	**Address**	**Data**
	0x10000000	f
	0x10000004	g
	0x10000008	y

```
addi $sp, $sp, –4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, –4
jr   $ra
add  $v0, $a0, $a1
jr   $ra
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: The Executable

Example (first store): sw $a0, 0x8000($gp) stores 2 into global f at 0x10000000.

- 0x8000 is a **16-bit signed** offset, sign-extended to 0xFFFF8000, then added to $gp:
- **$gp + 0x8000 = 0x10008000 + 0xFFFF8000 = 0x10000000** (address of f)

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	**Address**	**Instruction**
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	**Address**	**Data**
	0x10000000	f
	0x10000004	g
	0x10000008	y

```
addi $sp, $sp, –4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, –4
jr   $ra
add  $v0, $a0, $a1
jr   $ra
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 4 - Loading

The OS loads the program by:

1. Reading the **text segment** of the executable from storage (usually the hard disk) into the text segment of memory.
2. Setting **$gp = 0x10008000** (middle of the global data segment).
3. Setting **$sp = 0x7FFFFFFC** (top of the dynamic data segment).
4. Performing **jal 0x00400000** to jump to the start of the program.

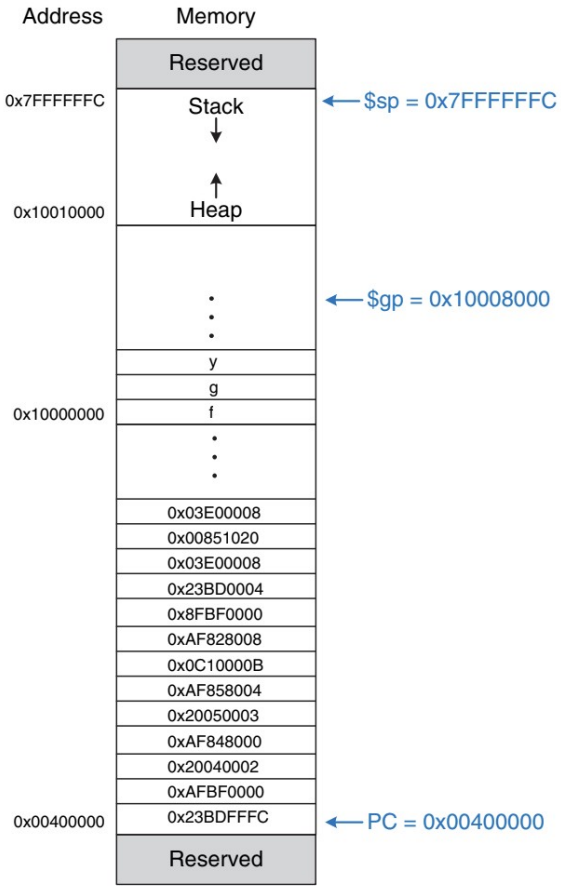
Memory at start of execution (Figure 6.34):

- Text segment from 0x00400000 holds the 13 instructions (0x23BDFFFC … 0x03E00008).
- f, g, y at 0x10000000–0x10000008 (global data).
- Stack at the top (0x7FFFFFFC), heap at the bottom of the dynamic segment (0x10010000).
- PC = 0x00400000.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Translating and Starting a Program: Step 4 - Loading



Executable loaded in memory

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q1. Which segment does the MIPS j instruction rely on being addressable directly, and why?

A. Stack, because it starts at 0x7FFFFFFC

B. Text, because the top 4 bits of its addresses are all 0

C. Heap, because it grows upward

D. Global data, because $gp is fixed

Q2. What is the value of $gp at program start, and what does it point to?

A. 0x10000000, the start of global data

B. 0x7FFFFFFC, the top of the stack

C. 0x10008000, the middle of the global data segment

D. 0x00400000, the start of the text segment

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q3. Why can any global variable be reached efficiently using $gp?

A. Globals are always stored in registers

B. The 16-bit signed offset from $gp is known at assembly time, so base addressing with a constant offset works

C. $gp changes with each function call to track the current variable

D. The linker converts all globals into immediate values

Q4. Which statement about the dynamic data segment is correct?

A. The heap grows downward from 0x7FFFFFFC and the stack grows upward

B. The stack grows downward from 0x7FFFFFFC and the heap grows upward from the bottom

C. Both grow downward, with the heap above the stack

D. It is reserved for the OS and the program cannot use it

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q5. What does the assembler do on its first pass?

A. Produces the final machine code

B. Relocates instructions from library files

C. Assigns instruction addresses and finds all symbols (labels and global variable names)

D. Loads the program into memory

Q6. In the example, sw $a0, 0x8000($gp) stores to which address?

A. 0x10008000 + 0x8000 = 0x10010000

B. 0x10008000 + 0xFFFF8000 = 0x10000000

C. 0x00008000

D. 0x7FFF8000

Q7. Which is the correct order of the steps that translate and start a program?

A. Assemble → Compile → Link → Load

B. Compile → Link → Assemble → Load

C. Compile → Assemble → Link → Load

D. Link → Compile → Assemble → Load

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Answer key

Q	Ans	Why
1	**B**	Top 4 bits of text addresses are 0, so j can jump to any program address.
2	**C**	The OS sets $gp to 0x10008000, the middle of the global data segment.
3	**B**	The offset is a signed 16-bit constant known at assembly time, so globals are accessed with base addressing.
4	**B**	The stack grows down from 0x7FFFFFFC, and the heap grows up from the bottom.
5	**C**	Pass 1 builds the symbol table. Pass 2 produces machine code.
6	**B**	The 0x8000 offset is sign-extended to 0xFFFF8000. Adding it to $gp gives 0x10000000, the address of f.
7	**C**	The pipeline is compiler → assembler → linker → loader.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)